

✓ Glass_vs_Plastic_Items_Classification

Student ID: 22F23626

Student Name: Shahla Suliman Alaamri

Module: Artificial Intelligence and Deep Learning

Assignment: Binary Image Classification using Deep Learning

Platform: Google Colab

Dataset/Topic: Glass vs Plastic Items - Kaggle

✓ Project Description

This project focuses on a binary image classification problem using deep learning. The selected topic is Glass vs Plastic Items classification. The main objective is to train a model that can classify whether an item in an image belongs to the Glass class or the Plastic class.

The project will be implemented using Python in Google Colab. A Convolutional Neural Network model will be built from scratch, and a pre-trained deep learning model will also be used for comparison. The performance of both models will be evaluated using classification metrics such as accuracy, precision, recall, and F1-score.

```
!pip install kaggle
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.3.0)
Requirement already satisfied: jupyter in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.19.3)
Requirement already satisfied: kagglesdk<1.0,>=0.1.20 in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.1.23)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kaggle) (26.2)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.6)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.9.0.post0)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.3)
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from bleach->kaggle) (0.5.1)
Requirement already satisfied: markdown-it-py>=1.0 in /usr/local/lib/python3.12/dist-packages (from jupyter->kaggle) (4.2.0)
Requirement already satisfied: mdit-py-plugins in /usr/local/lib/python3.12/dist-packages (from jupyter->kaggle) (0.6.1)
Requirement already satisfied: nbformat in /usr/local/lib/python3.12/dist-packages (from jupyter->kaggle) (5.10.4)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from jupyter->kaggle) (6.0.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil->kaggle) (1.17.0)
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.12/dist-packages (from python-slugify->kaggle) (1.3)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (3.15)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (2026.5.20)
Requirement already satisfied: mdurl~0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=1.0->jupyter->kaggle) (0.1.2)
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupyter->kaggle) (2.21.2)
Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupyter->kaggle) (4.26.0)
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupyter->kaggle) (5.9.1)
Requirement already satisfied: traitlets>=5.1 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupyter->kaggle) (5.7.1)
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->jupyter->kaggle) (26.1.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->jupyter->kaggle) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->jupyter->kaggle) (0.37.0)
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->jupyter->kaggle) (0.30.0)
```

Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.12/dist-packages (from jupyter-core!=5.0.*,>=4.12->nbformat->jupyter-text->kaggle) (4.9.6)
 Requirement already satisfied: typing-extensions>=4.4.0 in /usr/local/lib/python3.12/dist-packages (from referencing>=0.28.4->jsonschema>=2.6->nbformat->jupyter-text->kaggle) (4.1

```
from google.colab import files
files.upload()
```

Choose Files kaggle.json
 kaggle.json(application/json) - 66 bytes, last modified: 5/26/2026 - 100% done
 Saving kaggle.json to kaggle.json
 {'kaggle.json': b'{"username": "jabriomars", "key": "980987b3add0fafe9288e93c0e1d1a70"}'}

```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

```
!kaggle kernels pull supermarketthobo/glass-vs-plastic-items-image-classification
```

Source code downloaded to /content/glass-vs-plastic-items-image-classification.ipynb

```
import os

print(os.listdir())

['.config', 'glass-vs-plastic-items-image-classification.ipynb', 'kaggle.json', 'sample_data']
```

```
import os

for file in os.listdir():
    print(file)

.config
glass-vs-plastic-items-image-classification.ipynb
kaggle.json
sample_data
```

```
# Search inside the downloaded notebook for possible dataset names or paths
import json

notebook_path = "glass-vs-plastic-items-image-classification.ipynb"

with open(notebook_path, "r", encoding="utf-8") as f:
    notebook = json.load(f)

keywords = ["dataset", "glass", "plastic", "/kaggle/input", "train", "test", "ImageDataGenerator", "flow_from_directory"]

for cell in notebook["cells"]:
    source = "".join(cell.get("source", []))
    for keyword in keywords:
        if keyword.lower() in source.lower():
            print(source)
            print("-" * 100)
            break
```

batch_size=32.

```

class_mode='binary',
subset='training') # set as training data

validation_generator = image_generator.flow_from_directory(
    'images',
    target_size=(224, 224),
    batch_size=32,
    class_mode='binary',
    subset='validation') # set as validation data
-----
# Fit the model

# Source: https://stackoverflow.com/questions/42443936/keras-split-train-test-set-when-using-imagedatagenerator
batch_size=32

history = model.fit(
    train_generator,
    steps_per_epoch = train_generator.samples // batch_size,
    validation_data = validation_generator,
    validation_steps = validation_generator.samples // batch_size,
    callbacks = [early_stopping],
    epochs = 10)
-----
# See the model's training curves

history_df = pd.DataFrame(history.history)
# Start the plot at epoch 1
history_df.loc[0:, ['loss', 'val_loss']].plot()
history_df.loc[0:, ['binary_accuracy', 'val_binary_accuracy']].plot()

print(("Best Validation Loss: {:.4f}" + \
      "\nBest Validation Accuracy: {:.4f}")\
      .format(history_df['val_loss'].min(),
              history_df['val_binary_accuracy'].max()))
-----
# Comparison

Transfer learning with resnet18 got to about 90% accuracy. My CNN model got about 65% at most. The training curves for my model look very strange. Early stopping kicked in a
I tried reducing the number of filters and layers on my model which brings accuracy down to about 50%, which is chance level. Even with a lower capacity model, there was abo
Either case has way too many parameters for ~800 images to optimize from SCRATCH. Resnet models were trained on like 1,000,000+ images.
-----
#### Below are example training curves from my model using a cleaned dataset.
Validation accuracy barely changes but the validation loss converges towards the training data loss. The training data accuracy/loss is very poor in its own right. It seems


-----
# Areas for Improvement

* Download the data on my own machine so it is the same on each run, and cleaning is permanent. Right now I hardcode URLs and hardcode which images to delete/clean.
* Scrape more data so my own model won't underfit (even with data augmentation ~800 images wasn't enough to fully train a CNN)
* Try to find a model architecture with fewer parameters that performs just as well or better
* Can possibly productionize model on a toy website so other people can use it

```

```

# Install fastbook to download images from DuckDuckGo
!pip install fastbook

```

```

Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer>=0.20.0->huggingface-hub>=0.24.0->datasets->fastbook) (0.0.4)
Requirement already satisfied: cloudpathlib>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from weasel<2.0.0,>=1.0.0->spacy<4->fastai>=2.6->fastbook) (0.24.0)
Requirement already satisfied: smart-open>=5.2.1 in /usr/local/lib/python3.12/dist-packages (from weasel<2.0.0,>=1.0.0->spacy<4->fastai>=2.6->fastbook) (7.6.1)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->spacy<4->fastai>=2.6->fastbook) (3.0.3)
Requirement already satisfied: apswutils>=0.1.2 in /usr/local/lib/python3.12/dist-packages (from fastlite>=0.1.1->python-fasthtml>=0.12.34->fastprogress>=0.2.4->fastai>=2.6->fastbook) (0.1.2)
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.12/dist-packages (from jupyter-core>=4.9.2->jupyter-client>=6.1.12->ipykernel>=4.5.1->ipywidgets<8->fastbook) (4.1.2)
Requirement already satisfied: mdurl~0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=10.0.0->plum-dispatch->fastai>=2.6->fastbook) (0.1.2)
Requirement already satisfied: notebook-shim>=0.2.3 in /usr/local/lib/python3.12/dist-packages (from nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (0.2.3)
Requirement already satisfied: bleach!=5.0.0 in /usr/local/lib/python3.12/dist-packages (from bleach[css]!=5.0.0->nbconvert>=5->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (5.0.1)
Requirement already satisfied: defusedxml in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (0.7.1)
Requirement already satisfied: jupyterlab-pygments in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (0.3.0)
Requirement already satisfied: mistune<4,>=2.0.3 in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (3.0.2)
Requirement already satisfied: nbclient>=0.5.0 in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (0.8.0)
Requirement already satisfied: pandocfilters>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from nbconvert>=5->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (1.5.1)
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.12/dist-packages (from nbformat>=2.0.0->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (2.20.0)
Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.12/dist-packages (from nbformat>=2.0.0->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (4.21.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open>=5.2.1->weasel<2.0.0,>=1.0.0->spacy<4->fastai>=2.6->fastbook) (2.2.0)
Requirement already satisfied: httptools>=0.6.3 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standard]>=0.30->python-fasthtml>=0.12.34->fastprogress>=0.2.4->fastbook) (0.4.0)
Requirement already satisfied: python-dotenv>=0.13 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standard]>=0.30->python-fasthtml>=0.12.34->fastprogress>=0.2.4->fastbook) (0.21.0)
Requirement already satisfied: uvloop>=0.15.1 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standard]>=0.30->python-fasthtml>=0.12.34->fastprogress>=0.2.4->fastbook) (0.21.0)
Requirement already satisfied: watchfiles>=0.20 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standard]>=0.30->python-fasthtml>=0.12.34->fastprogress>=0.2.4->fastbook) (1.0.4)
Requirement already satisfied: websockets>=10.4 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standard]>=0.30->python-fasthtml>=0.12.34->fastprogress>=0.2.4->fastbook) (13.1)
Requirement already satisfied: argon2-cffi-bindings in /usr/local/lib/python3.12/dist-packages (from argon2-cffi->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (21.1.0)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->python-fasthtml>=0.12.34->fastprogress>=0.2.4->fastbook) (2.5)
Requirement already satisfied: apsw in /usr/local/lib/python3.12/dist-packages (from apswutils>=0.1.2->fastlite>=0.1.1->python-fasthtml>=0.12.34->fastprogress>=0.2.4->fastbook) (3.45.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconvert>=5->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (0.11)
Requirement already satisfied: tinycss2<1.5,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from bleach[css]!=5.0.0->nbconvert>=5->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (1.3.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat>=2.0.0->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (2023.12.1)
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat>=2.0.0->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (0.35.0)
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat>=2.0.0->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (0.20.0)
Requirement already satisfied: jupyter-server<3,>=1.8 in /usr/local/lib/python3.12/dist-packages (from notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (2.14.1)
Requirement already satisfied: cffi>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from argon2-cffi-bindings->argon2-cffi->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (1.17.1)
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cffi>=1.0.1->argon2-cffi-bindings->argon2-cffi->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (2.22)
Requirement already satisfied: jupyter-events>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (0.10.0)
Requirement already satisfied: jupyter-server-terminals>=0.4.4 in /usr/local/lib/python3.12/dist-packages (from jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (0.5.3)
Requirement already satisfied: overrides>=5.0 in /usr/local/lib/python3.12/dist-packages (from jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (7.7.0)
Requirement already satisfied: websocket-client>=1.7 in /usr/local/lib/python3.12/dist-packages (from jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (1.7.0)
Requirement already satisfied: python-json-logger>=2.0.4 in /usr/local/lib/python3.12/dist-packages (from jupyter-events>=0.9.0->jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (3.0.0)
Requirement already satisfied: rfc3339-validator in /usr/local/lib/python3.12/dist-packages (from jupyter-events>=0.9.0->jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (0.1.4)
Requirement already satisfied: rfc3986-validator>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from jupyter-events>=0.9.0->jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (0.4.0)
Requirement already satisfied: fqdn in /usr/local/lib/python3.12/dist-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.9.0->jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (1.5.1)
Requirement already satisfied: isoduration in /usr/local/lib/python3.12/dist-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.9.0->jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (2.1.0)
Requirement already satisfied: jsonpointer>1.13 in /usr/local/lib/python3.12/dist-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.9.0->jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (2.3)
Requirement already satisfied: rfc3987-syntax>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.9.0->jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (0.1.0)
Requirement already satisfied: uri-template in /usr/local/lib/python3.12/dist-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.9.0->jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (0.1.0)
Requirement already satisfied: webcolors>=24.6.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.9.0->jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (24.6.0)
Requirement already satisfied: lark>=1.2.2 in /usr/local/lib/python3.12/dist-packages (from rfc3987-syntax>=1.1.0->jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.9.0->jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (1.2.2)
Requirement already satisfied: arrow>=0.15.0 in /usr/local/lib/python3.12/dist-packages (from isoduration->jsonschema[format-nongpl]>=4.18.0->jupyter-events>=0.9.0->jupyter-server<3,>=1.8->notebook-shim>=0.2.3->nbclassic>=0.4.7->notebook>=4.4.1->widgetsnbextension~3.6.0->ipywidgets<8->fastbook) (1.3.0)
Downloading fastbook-0.0.29-py3-none-any.whl (719 kB)

```

719.8/719.8 kB 15.7 MB/s eta 0:00:00

Downloading jedi-0.20.0-py2.py3-none-any.whl (4.9 MB)

4.9/4.9 MB 75.6 MB/s eta 0:00:00

Installing collected packages: jedi, fastbook

```

# Import required libraries
from fastbook import *
from fastai.vision.all import *
import os
from pathlib import Path

```

```
# Create dataset folders
path = Path("glass_plastic_dataset")

glass_path = path / "glass"
plastic_path = path / "plastic"

glass_path.mkdir(parents=True, exist_ok=True)
plastic_path.mkdir(parents=True, exist_ok=True)

print("Dataset folders created successfully.")
```

Dataset folders created successfully.

```
# Extract hard-coded image URLs from the downloaded Kaggle notebook
import json
import re
import ast

notebook_path = "glass-vs-plastic-items-image-classification.ipynb"

with open(notebook_path, "r", encoding="utf-8") as f:
    notebook = json.load(f)

all_code = "\n".join(
    "".join(cell.get("source", []))
    for cell in notebook["cells"]
    if cell.get("cell_type") == "code"
)

def extract_url_list(variable_name):
    pattern = rf"{variable_name}\s*=\s*(\[.*?\])"
    match = re.search(pattern, all_code, re.DOTALL)
    if match:
        return ast.literal_eval(match.group(1))
    return []

glass_urls = extract_url_list("glass_urls")
plastic_urls = extract_url_list("plastic_urls")

print("Glass URLs found:", len(glass_urls))
print("Plastic URLs found:", len(plastic_urls))
```

Glass URLs found: 480
Plastic URLs found: 467

```
# Create dataset folders
import os
from pathlib import Path

path = Path("glass_plastic_dataset")
glass_path = path / "glass"
plastic_path = path / "plastic"

glass_path.mkdir(parents=True, exist_ok=True)
plastic_path.mkdir(parents=True, exist_ok=True)

print("Folders created successfully.")
```

Folders created successfully.

```
# Download images safely with timeout
import requests
from PIL import Image
from io import BytesIO

def safe_download_images(urls, folder, limit=40):
    count = 0

    for i, url in enumerate(urls[:limit]):
        try:
            response = requests.get(url, timeout=8)

            if response.status_code == 200:
                img = Image.open(BytesIO(response.content)).convert("RGB")
                img.save(folder / f"image_{i}.jpg")
                count += 1

        except Exception:
            pass

    return count

glass_count = safe_download_images(glass_urls, glass_path, limit=40)
plastic_count = safe_download_images(plastic_urls, plastic_path, limit=40)

print("Glass images downloaded:", glass_count)
print("Plastic images downloaded:", plastic_count)
```

```
/usr/local/lib/python3.12/dist-packages/PIL/Image.py:1047: UserWarning: Palette images with Transparency expressed in bytes should be converted to RGBA images
  warnings.warn(
Glass images downloaded: 26
Plastic images downloaded: 25
```

```
# Download more images from later URLs to increase dataset size
def safe_download_more_images(urls, folder, start=40, end=120, prefix="more"):
    count = 0

    for i, url in enumerate(urls[start:end]):
        try:
            response = requests.get(url, timeout=8)

            if response.status_code == 200:
                img = Image.open(BytesIO(response.content)).convert("RGB")
                img.save(folder / f"{prefix}_{start+i}.jpg")
                count += 1

        except Exception:
            pass

    return count

more_glass_count = safe_download_more_images(glass_urls, glass_path, start=40, end=120, prefix="glass")
more_plastic_count = safe_download_more_images(plastic_urls, plastic_path, start=40, end=120, prefix="plastic")
```

```
print("More glass images downloaded:", more_glass_count)
print("More plastic images downloaded:", more_plastic_count)
```

```
More glass images downloaded: 48
More plastic images downloaded: 44
```

```
# Check updated final number of images
glass_count = len(list(glass_path.glob("*")))
plastic_count = len(list(plastic_path.glob("*")))

print("Number of glass images:", glass_count)
print("Number of plastic images:", plastic_count)
print("Total images:", glass_count + plastic_count)
```

```
Number of glass images: 74
Number of plastic images: 69
Total images: 143
```

```
# Display sample images from the dataset
import matplotlib.pyplot as plt
from PIL import Image
import random

image_files = list(path.glob("*/*"))
sample_images = random.sample(image_files, 6)

plt.figure(figsize=(12, 6))

for i, img_path in enumerate(sample_images, 1):
    img = Image.open(img_path)

    plt.subplot(2, 3, i)
    plt.imshow(img)
    plt.axis("off")
    plt.title(img_path.parent.name)

plt.tight_layout()
plt.show()
```



```
# Import ImageDataGenerator
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Image settings
IMG_SIZE = 224
BATCH_SIZE = 16

# Training generator with augmentation
train_generator = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2,
    rotation_range=15,
    zoom_range=0.1,
    horizontal_flip=True
)

# Validation generator without augmentation
val_generator = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)
```

```
# Load training images
train_images = train_generator.flow_from_directory(
```

```
    path,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="binary",
    subset="training",
    shuffle=True
)

# Load validation images
val_images = val_generator.flow_from_directory(
    path,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="binary",
    subset="validation",
    shuffle=False
)
```

```
Found 116 images belonging to 2 classes.
Found 27 images belonging to 2 classes.
```

```
# Check class indices
print("Class indices:")
print(train_images.class_indices)
```

```
Class indices:
{'glass': 0, 'plastic': 1}
```

```
# Import TensorFlow and Keras layers
import tensorflow as tf
from tensorflow.keras import layers, models
```

```
# Build CNN model from scratch
cnn_model = models.Sequential([
    layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3)),

    # First convolution block
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),

    # Second convolution block
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),

    # Third convolution block
    layers.Conv2D(128, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),

    # Convert feature maps into one vector
    layers.Flatten(),

    # Fully connected layer
    layers.Dense(128, activation="relu"),

    # Dropout to reduce overfitting
    layers.Dropout(0.5),
```

```
# Output layer for binary classification
layers.Dense(1, activation="sigmoid")
])

# Show model architecture
cnn_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten (Flatten)	(None, 86528)	0
dense (Dense)	(None, 128)	11,075,712
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129

Total params: 11,169,089 (42.61 MB)
 Trainable params: 11,169,089 (42.61 MB)
 Non-trainable params: 0 (0.00 B)

```
# Compile CNN model
cnn_model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

```
# Train CNN model
history_cnn = cnn_model.fit(
    train_images,
    validation_data=val_images,
    epochs=10
)
```

```
Epoch 1/10
8/8 ————— 18s 2s/step - accuracy: 0.4138 - loss: 1.5475 - val_accuracy: 0.5185 - val_loss: 0.7097
Epoch 2/10
8/8 ————— 15s 2s/step - accuracy: 0.5172 - loss: 0.6998 - val_accuracy: 0.5926 - val_loss: 0.6787
Epoch 3/10
8/8 ————— 20s 2s/step - accuracy: 0.5690 - loss: 0.6922 - val_accuracy: 0.6667 - val_loss: 0.6788
Epoch 4/10
8/8 ————— 14s 2s/step - accuracy: 0.6034 - loss: 0.6758 - val_accuracy: 0.5185 - val_loss: 0.6755
Epoch 5/10
8/8 ————— 16s 2s/step - accuracy: 0.6293 - loss: 0.6447 - val_accuracy: 0.5185 - val_loss: 0.7025
Epoch 6/10
8/8 ————— 18s 2s/step - accuracy: 0.6897 - loss: 0.6335 - val_accuracy: 0.6296 - val_loss: 0.7015
```

```
Epoch 7/10
8/8 ————— 14s 2s/step - accuracy: 0.6724 - loss: 0.6242 - val_accuracy: 0.4815 - val_loss: 0.7813
Epoch 8/10
8/8 ————— 14s 2s/step - accuracy: 0.6724 - loss: 0.5956 - val_accuracy: 0.6667 - val_loss: 0.7419
Epoch 9/10
8/8 ————— 14s 2s/step - accuracy: 0.7586 - loss: 0.5478 - val_accuracy: 0.4815 - val_loss: 1.1128
Epoch 10/10
8/8 ————— 14s 2s/step - accuracy: 0.7241 - loss: 0.6054 - val_accuracy: 0.4815 - val_loss: 0.7455
```

```
# Evaluate CNN model on validation data
cnn_loss, cnn_accuracy = cnn_model.evaluate(val_images)
```

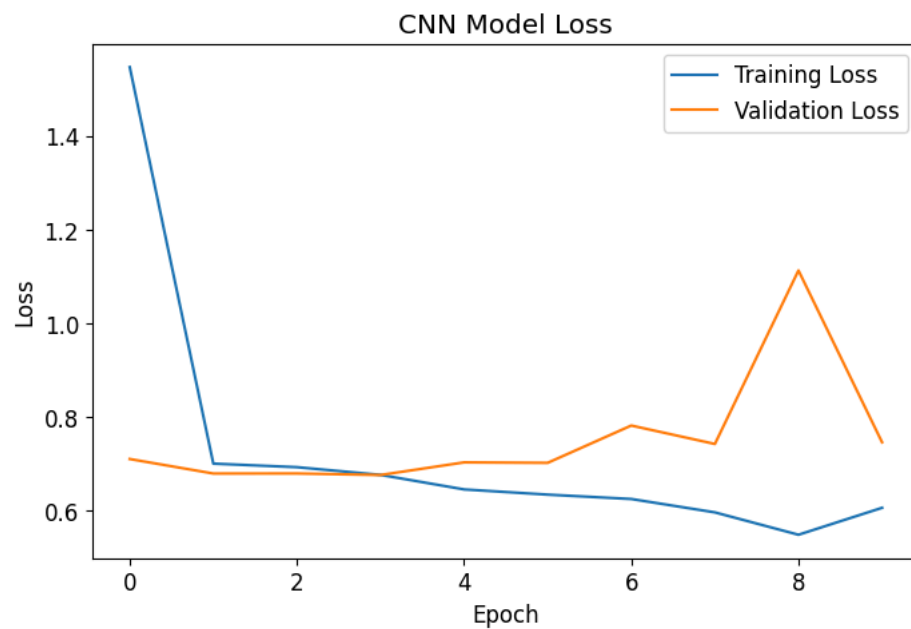
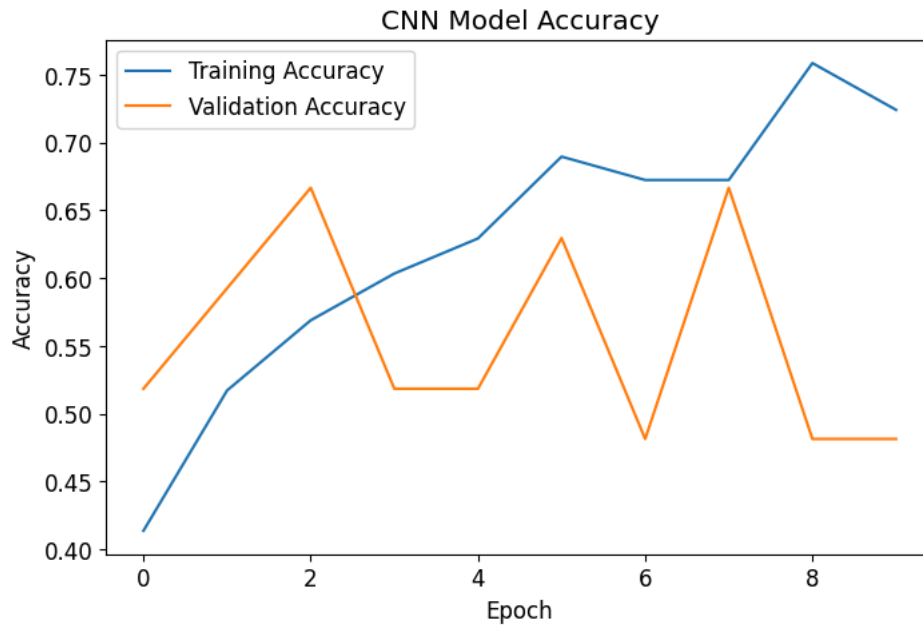
```
print("CNN Validation Loss:", cnn_loss)
print("CNN Validation Accuracy:", cnn_accuracy)
```

```
2/2 ————— 2s 837ms/step - accuracy: 0.4815 - loss: 0.7455
CNN Validation Loss: 0.7454943060874939
CNN Validation Accuracy: 0.48148149251937866
```

```
# Plot CNN training and validation accuracy
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.plot(history_cnn.history["accuracy"], label="Training Accuracy")
plt.plot(history_cnn.history["val_accuracy"], label="Validation Accuracy")
plt.title("CNN Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# Plot CNN training and validation loss
plt.figure(figsize=(8, 5))
plt.plot(history_cnn.history["loss"], label="Training Loss")
plt.plot(history_cnn.history["val_loss"], label="Validation Loss")
plt.title("CNN Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```



```
# Build a smaller CNN model to reduce overfitting
cnn_model = models.Sequential([
    layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3)),

    layers.Conv2D(16, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
```

```

layers.Conv2D(32, (3, 3), activation="relu"),
layers.MaxPooling2D((2, 2)),

layers.Conv2D(64, (3, 3), activation="relu"),
layers.MaxPooling2D((2, 2)),

layers.GlobalAveragePooling2D(),

layers.Dense(64, activation="relu"),
layers.Dropout(0.5),

layers.Dense(1, activation="sigmoid")
])

cnn_model.summary()

```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 222, 222, 16)	448
max_pooling2d_3 (MaxPooling2D)	(None, 111, 111, 16)	0
conv2d_4 (Conv2D)	(None, 109, 109, 32)	4,640
max_pooling2d_4 (MaxPooling2D)	(None, 54, 54, 32)	0
conv2d_5 (Conv2D)	(None, 52, 52, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 26, 26, 64)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 64)	0
dense_2 (Dense)	(None, 64)	4,160
dropout_1 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 1)	65

Total params: 27,809 (108.63 KB)
 Trainable params: 27,809 (108.63 KB)
 Non-trainable params: 0 (0.00 B)

```

cnn_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

```

```

history_cnn = cnn_model.fit(
    train_images,
    validation_data=val_images,
    epochs=15
)

```

Epoch 1/15
 8/8 — 11s 1s/step - accuracy: 0.4828 - loss: 0.7008 - val_accuracy: 0.4815 - val_loss: 0.6933

```

Epoch 2/15
8/8 ————— 8s 983ms/step - accuracy: 0.4914 - loss: 0.6919 - val_accuracy: 0.4815 - val_loss: 0.6931
Epoch 3/15
8/8 ————— 9s 779ms/step - accuracy: 0.4828 - loss: 0.6930 - val_accuracy: 0.4815 - val_loss: 0.6929
Epoch 4/15
8/8 ————— 8s 1s/step - accuracy: 0.4569 - loss: 0.6959 - val_accuracy: 0.4815 - val_loss: 0.6928
Epoch 5/15
8/8 ————— 7s 815ms/step - accuracy: 0.5431 - loss: 0.6880 - val_accuracy: 0.4815 - val_loss: 0.6928
Epoch 6/15
8/8 ————— 8s 947ms/step - accuracy: 0.5086 - loss: 0.6945 - val_accuracy: 0.4815 - val_loss: 0.6926
Epoch 7/15
8/8 ————— 8s 914ms/step - accuracy: 0.5259 - loss: 0.6880 - val_accuracy: 0.4815 - val_loss: 0.6923
Epoch 8/15
8/8 ————— 7s 836ms/step - accuracy: 0.4483 - loss: 0.6972 - val_accuracy: 0.4815 - val_loss: 0.6920
Epoch 9/15
8/8 ————— 9s 1s/step - accuracy: 0.4741 - loss: 0.6910 - val_accuracy: 0.4815 - val_loss: 0.6918
Epoch 10/15
8/8 ————— 10s 961ms/step - accuracy: 0.4741 - loss: 0.6971 - val_accuracy: 0.4815 - val_loss: 0.6917
Epoch 11/15
8/8 ————— 8s 858ms/step - accuracy: 0.5172 - loss: 0.6941 - val_accuracy: 0.4815 - val_loss: 0.6914
Epoch 12/15
8/8 ————— 9s 1s/step - accuracy: 0.5000 - loss: 0.6954 - val_accuracy: 0.4815 - val_loss: 0.6912
Epoch 13/15
8/8 ————— 7s 797ms/step - accuracy: 0.4914 - loss: 0.6910 - val_accuracy: 0.4815 - val_loss: 0.6908
Epoch 14/15
8/8 ————— 8s 988ms/step - accuracy: 0.5259 - loss: 0.6895 - val_accuracy: 0.4815 - val_loss: 0.6904
Epoch 15/15
8/8 ————— 9s 1s/step - accuracy: 0.4914 - loss: 0.6940 - val_accuracy: 0.4815 - val_loss: 0.6896

```

```
cnn_loss, cnn_accuracy = cnn_model.evaluate(val_images)
```

```
print("CNN Validation Loss:", cnn_loss)
print("CNN Validation Accuracy:", cnn_accuracy)
```

```

2/2 ————— 1s 257ms/step - accuracy: 0.4815 - loss: 0.6896
CNN Validation Loss: 0.6895580887794495
CNN Validation Accuracy: 0.48148149251937866

```

```

# Import MobileNetV2 pre-trained model
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

```

```

# Data generators for MobileNetV2
mobilenet_train_generator = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    validation_split=0.2,
    rotation_range=20,
    zoom_range=0.2,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)

mobilenet_val_generator = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    validation_split=0.2
)

```

```
# Load MobileNetV2 training images
mobilenet_train_images = mobilenet_train_generator.flow_from_directory(
    path,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="binary",
    subset="training",
    shuffle=True
)

# Load MobileNetV2 validation images
mobilenet_val_images = mobilenet_val_generator.flow_from_directory(
    path,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="binary",
    subset="validation",
    shuffle=False
)

print("Class indices:")
print(mobilenet_train_images.class_indices)
```

```
Found 116 images belonging to 2 classes.
Found 27 images belonging to 2 classes.
Class indices:
{'glass': 0, 'plastic': 1}
```

```
# Load MobileNetV2 base model
base_model = MobileNetV2(
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    include_top=False,
    weights="imagenet"
)

# Freeze base model layers
base_model.trainable = False

# Build transfer learning model
mobilenet_model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.BatchNormalization(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(1, activation="sigmoid")
])

mobilenet_model.summary()
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
 9406464/9406464 ————— 0s 0us/step

Model: "sequential_2"

Layer (type)	Output Shape	Param #
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
batch_normalization (BatchNormalization)	(None, 1280)	5,120
dense_4 (Dense)	(None, 128)	163,968
dropout_2 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 1)	129

Total params: 2,427,201 (9.26 MB)

Trainable params: 166,657 (651.00 KB)

Non-trainable params: 2,260,544 (8.62 MB)

```
# Compile MobileNetV2 model
mobilenet_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

```
# Callbacks to improve training
early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

reduce_lr = ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.2,
    patience=3,
    min_lr=0.000001
)
```

```
# Train MobileNetV2 model
history_mobilenet = mobilenet_model.fit(
    mobilenet_train_images,
    validation_data=mobilenet_val_images,
    epochs=20,
    callbacks=[early_stopping, reduce_lr]
)
```

Epoch 1/20

8/8 ————— 15s 1s/step - accuracy: 0.4914 - loss: 1.0211 - val_accuracy: 0.4074 - val_loss: 0.7961 - learning_rate: 1.0000e-04

Epoch 2/20

8/8 ————— 8s 920ms/step - accuracy: 0.5086 - loss: 0.8850 - val_accuracy: 0.4815 - val_loss: 0.7595 - learning_rate: 1.0000e-04

```

Epoch 3/20
8/8 ————— 7s 818ms/step - accuracy: 0.6552 - loss: 0.7447 - val_accuracy: 0.5926 - val_loss: 0.7287 - learning_rate: 1.0000e-04
Epoch 4/20
8/8 ————— 10s 743ms/step - accuracy: 0.6810 - loss: 0.6088 - val_accuracy: 0.5926 - val_loss: 0.6967 - learning_rate: 1.0000e-04
Epoch 5/20
8/8 ————— 8s 953ms/step - accuracy: 0.6897 - loss: 0.6382 - val_accuracy: 0.6296 - val_loss: 0.6726 - learning_rate: 1.0000e-04
Epoch 6/20
8/8 ————— 6s 755ms/step - accuracy: 0.7155 - loss: 0.6146 - val_accuracy: 0.6667 - val_loss: 0.6497 - learning_rate: 1.0000e-04
Epoch 7/20
8/8 ————— 8s 918ms/step - accuracy: 0.7759 - loss: 0.5131 - val_accuracy: 0.6667 - val_loss: 0.6357 - learning_rate: 1.0000e-04
Epoch 8/20
8/8 ————— 8s 1s/step - accuracy: 0.7500 - loss: 0.5499 - val_accuracy: 0.6667 - val_loss: 0.6279 - learning_rate: 1.0000e-04
Epoch 9/20
8/8 ————— 9s 804ms/step - accuracy: 0.8103 - loss: 0.5303 - val_accuracy: 0.6667 - val_loss: 0.6226 - learning_rate: 1.0000e-04
Epoch 10/20
8/8 ————— 10s 1s/step - accuracy: 0.7845 - loss: 0.4631 - val_accuracy: 0.7037 - val_loss: 0.6225 - learning_rate: 1.0000e-04
Epoch 11/20
8/8 ————— 8s 952ms/step - accuracy: 0.7931 - loss: 0.4419 - val_accuracy: 0.7407 - val_loss: 0.6229 - learning_rate: 1.0000e-04
Epoch 12/20
8/8 ————— 6s 770ms/step - accuracy: 0.7931 - loss: 0.4654 - val_accuracy: 0.7407 - val_loss: 0.6235 - learning_rate: 1.0000e-04
Epoch 13/20
8/8 ————— 8s 960ms/step - accuracy: 0.8362 - loss: 0.3607 - val_accuracy: 0.7407 - val_loss: 0.6259 - learning_rate: 2.0000e-05
Epoch 14/20
8/8 ————— 7s 786ms/step - accuracy: 0.8534 - loss: 0.3773 - val_accuracy: 0.7407 - val_loss: 0.6265 - learning_rate: 2.0000e-05
Epoch 15/20
8/8 ————— 8s 993ms/step - accuracy: 0.8707 - loss: 0.3072 - val_accuracy: 0.7407 - val_loss: 0.6298 - learning_rate: 2.0000e-05

```

```

# Evaluate MobileNetV2 model
mobilenet_loss, mobilenet_accuracy = mobilenet_model.evaluate(mobilenet_val_images)

print("MobileNetV2 Validation Loss:", mobilenet_loss)
print("MobileNetV2 Validation Accuracy:", mobilenet_accuracy)

2/2 ————— 2s 524ms/step - accuracy: 0.7037 - loss: 0.6225
MobileNetV2 Validation Loss: 0.6224715709686279
MobileNetV2 Validation Accuracy: 0.7037037014961243

```

```

# Fine-tune the last layers of MobileNetV2
base_model.trainable = True

# Freeze most layers and train only the last 30 layers
for layer in base_model.layers[:-30]:
    layer.trainable = False

# Recompile with a very small learning rate
mobilenet_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.00001),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

# Continue training with fine-tuning
history_finetune = mobilenet_model.fit(
    mobilenet_train_images,
    validation_data=mobilenet_val_images,
    epochs=10,
    callbacks=[early_stopping, reduce_lr]
)

```

```

Epoch 1/10
8/8 ————— 24s 1s/step - accuracy: 0.7155 - loss: 0.6165 - val_accuracy: 0.7407 - val_loss: 0.6139 - learning_rate: 1.0000e-05
Epoch 2/10
8/8 ————— 9s 1s/step - accuracy: 0.7672 - loss: 0.4687 - val_accuracy: 0.7037 - val_loss: 0.6078 - learning_rate: 1.0000e-05
Epoch 3/10
8/8 ————— 8s 981ms/step - accuracy: 0.7069 - loss: 0.5703 - val_accuracy: 0.7407 - val_loss: 0.6004 - learning_rate: 1.0000e-05
Epoch 4/10
8/8 ————— 10s 993ms/step - accuracy: 0.7672 - loss: 0.4650 - val_accuracy: 0.7407 - val_loss: 0.5944 - learning_rate: 1.0000e-05
Epoch 5/10
8/8 ————— 9s 1s/step - accuracy: 0.7672 - loss: 0.4722 - val_accuracy: 0.7407 - val_loss: 0.5898 - learning_rate: 1.0000e-05
Epoch 6/10
8/8 ————— 9s 1s/step - accuracy: 0.8103 - loss: 0.4279 - val_accuracy: 0.7407 - val_loss: 0.5867 - learning_rate: 1.0000e-05
Epoch 7/10
8/8 ————— 8s 988ms/step - accuracy: 0.8017 - loss: 0.4011 - val_accuracy: 0.7778 - val_loss: 0.5837 - learning_rate: 1.0000e-05
Epoch 8/10
8/8 ————— 9s 1s/step - accuracy: 0.7759 - loss: 0.4882 - val_accuracy: 0.7407 - val_loss: 0.5813 - learning_rate: 1.0000e-05
Epoch 9/10
8/8 ————— 9s 1s/step - accuracy: 0.7759 - loss: 0.4148 - val_accuracy: 0.7407 - val_loss: 0.5785 - learning_rate: 1.0000e-05
Epoch 10/10
8/8 ————— 8s 962ms/step - accuracy: 0.8017 - loss: 0.4150 - val_accuracy: 0.7407 - val_loss: 0.5768 - learning_rate: 1.0000e-05

```

```

# Evaluate fine-tuned MobileNetV2 model
finetuned_loss, finetuned_accuracy = mobilenet_model.evaluate(mobilenet_val_images)

print("Fine-tuned MobileNetV2 Validation Loss:", finetuned_loss)
print("Fine-tuned MobileNetV2 Validation Accuracy:", finetuned_accuracy)

```

```

2/2 ————— 1s 358ms/step - accuracy: 0.7407 - loss: 0.5768
Fine-tuned MobileNetV2 Validation Loss: 0.5767804384231567
Fine-tuned MobileNetV2 Validation Accuracy: 0.7407407164573669

```

```

# Create comparison table
import pandas as pd

comparison_df = pd.DataFrame({
    "Model": [
        "CNN from Scratch",
        "MobileNetV2",
        "Fine-tuned MobileNetV2"
    ],
    "Validation Loss": [
        cnn_loss,
        mobilenet_loss,
        finetuned_loss
    ],
    "Validation Accuracy": [
        cnn_accuracy,
        mobilenet_accuracy,
        finetuned_accuracy
    ]
})

display(comparison_df)

```

	Model	Validation Loss	Validation Accuracy	
0	CNN from Scratch	0.689558	0.481481	
1	MobileNetV2	0.622472	0.703704	
2	Fine-tuned MobileNetV2	0.576780	0.740741	

Next steps:

[Generate code with comparison_df](#)[New interactive sheet](#)

```
# Classification report for Fine-tuned MobileNetV2
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

mobilenet_val_images.reset()

predictions = mobilenet_model.predict(mobilenet_val_images)
predicted_classes = (predictions > 0.5).astype("int32").flatten()

true_classes = mobilenet_val_images.classes
class_names = list(mobilenet_val_images.class_indices.keys())

print("Fine-tuned MobileNetV2 Classification Report:")
print(classification_report(true_classes, predicted_classes, target_names=class_names))
```

2/2  9s 3s/step

Fine-tuned MobileNetV2 Classification Report:

	precision	recall	f1-score	support
glass	0.71	0.86	0.77	14
plastic	0.80	0.62	0.70	13
accuracy			0.74	27
macro avg	0.75	0.74	0.73	27
weighted avg	0.75	0.74	0.74	27

```
# Confusion matrix for Fine-tuned MobileNetV2
cm = confusion_matrix(true_classes, predicted_classes)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=class_names
)

disp.plot()
plt.title("Fine-tuned MobileNetV2 Confusion Matrix")
plt.show()
```

Fine-tuned MobileNetV2 Confusion Matrix

